04 – USART

1. Cos'è la USART?

USART = Universal Synchronous/Asynchronous Receiver/Transmitter.
È una periferica del microcontrollore che permette di **trasmettere e ricevere dati serialmente**.
I dati vengono inviati **un bit alla volta** attraverso una linea seriale.
La USART può funzionare in:

- **Asynchronous** → comunicazione asincrona, tipicamente chiamata UART
- **Synchronous** → comunicazione sincrona, con anche un **clock**

Nella pratica, quando usi una Nucleo per comunicare con il PC, molto spesso utilizzi la modalità **asynchronous**.

2. USART vs UART

La distinzione importante è:

- **UART**→ solo comunicazione asincrona.
- **USART**→ può fare sia comunicazione asincrona sia sincrona.

Quindi una USART configurata in modalità asincrona si comporta sostanzialmente come una UART.

3. Come funziona la comunicazione seriale?

Supponiamo di voler mandare: "A"
Il carattere A in ASCII vale: A = 0x41 = 01000001

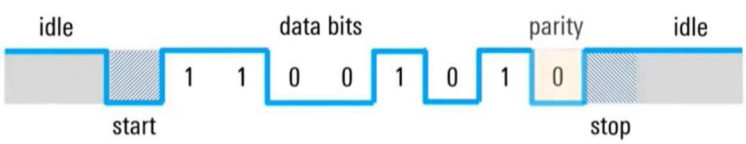
La USART non manda semplicemente questi 8 bit. Costruisce una **frame seriale**, ad esempio:

```
START     DATA        STOP
  ↓       ↓           ↓
  0    01000001       1
  |_____________________|
        frame
```

4. Il frame

- 1 start bit = 0
- 8 data bit = b0, b1, b2, b3, b4, b5, b6, b6
- Even/Odd Parity (opzionale, per error detection)
- 1 stop bit = 1
- Idle = 1

Una cosa importante: **i bit dei dati vengono normalmente trasmessi LSB first**, cioè dal bit meno significativo al più significativo.



5. Cosa significa "baud rate"?

È la velocità della comunicazione. Ad esempio 115200 baud significa che vengono trasmessi circa **115200 simboli al secondo**. Nel caso di una UART con un bit per simbolo, possiamo intuitivamente pensare a 1 bit ≈ 1 / 115200 s, quindi circa 8.68 µs/bit.

Attenzione però: **baud rate e bit rate non sono sempre la stessa cosa** in generale; nella UART semplice, con un bit per simbolo, coincidono numericamente.

6. Perché servono START e STOP?

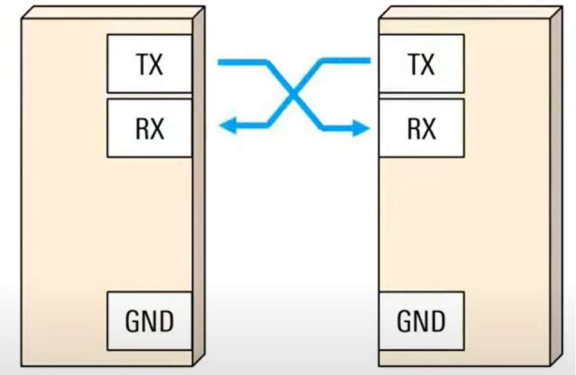
La comunicazione è **asincrona**. Questo significa che TX e RX **non condividono un clock**. Quindi il ricevitore deve capire: "Quando inizia un nuovo byte?" Lo fa grazie allo **start bit**.
In Idle la linea è alta (1), quando arriva (0) il ricevitore riconosce lo **start bit** e inizia a campionare i dati secondo il baud rate configurato. Alla fine trova lo **stop bit**, che normalmente vale (1).

7. TX e RX

La USART ha principalmente due linee:

- **TX — Transmit**
  - È l'uscita: STM32 TX –> dispositivo RX
- **RX — Receive**
  - È l'ingresso: STM32 RX <– dispositivo TX

Quindi bisogna incrociarle.



8. E sulla Nucleo?

Qui c'è una cosa molto utile da capire. Su molte Nucleo, il microcontrollore è collegato al **debugger ST-LINK**, che può fornire anche una **Virtual COM Port**. Quindi puoi scrivere dal microcontrollore HAL_UART_Transmit(...) e vedere il testo sul PC tramite una **serial terminal**.

9. Configurazione con STM32CubeMX

In STM32CubeIDE apri il file .ioc. Nella configurazione delle periferiche puoi trovare, ad esempio USART2 e impostarla come:

- Mode: Asynchronous

CubeMX configura automaticamente diversi parametri.

Per esempio:

- Baud Rate: 115200 Bits/s
- Word Length: 8 Bits
- Parity: None
- Stop Bits: 1
- Hardware Flow: None

10. Trasmettere con HAL

Una delle funzioni più importanti è:
HAL_UART_Transmit(&huart2,data,size,timeout);

Per esempio:

```
uint8_t message[] = "Hello!\r\n";

HAL_UART_Transmit(&huart2,message,sizeof(message)-1,HAL_MAX_DELAY);
```

La periferica prende i caratteri:H e l l o ! \r \n e li converte in una sequenza di frame seriali.

11. Ricevere dati

Per ricevere possiamo usare:

HAL_UART_Receive(&huart2,data,size,timeout);

Ad esempio:

```
uint8_t data;

HAL_UART_Receive(&huart2,&data,1,HAL_MAX_DELAY);
```

Il microcontrollore rimane in attesa finché non riceve un byte. Se dal PC invii "A", allora `data` conterrà "A".

12. Polling, Interrupt e DMA

La stessa USART può essere utilizzata in tre modi principali:

1. **Polling**: Il programma controlla direttamente lo stato della periferica. È il metodo più semplice.
2. **Interrupt**: La USART genera un **interrupt** quando avviene un determinato evento. Ad esempio, quando ha ricevuto un dato. Il programma principale può quindi continuare a fare altro.
3. **DMA**: Il trasferimento dei dati viene gestito dal **DMA**, senza che la CPU debba occuparsi di ogni singolo byte. È particolarmente utile quando devi trasferire molti dati. **NB: Quando usi DMA devi abilitare la Global Interrupt, altrimenti rimane HAL_BUSY.**

13. Un concetto fondamentale: USART è una periferica

Non pensare alla USART come a "un protocollo" in senso stretto. È soprattutto un **blocco hardware del microcontrollore** che si occupa di:

- serializzare i dati;
- aggiungere start/stop bit;
- generare il timing;
- trasmettere;
- ricevere;
- controllare eventuale parity;
- segnalare eventi tramite flag/interrupt.

14. USART vs I^2C vs SPI

Per fissare le idee:

	USART/UART	SPI	I²C
Clock	No*	Sì	Sì
Linee principali	TX, RX	SCK, MOSI, MISO, CS	SDA, SCL
Comunicazione	seriale	seriale	seriale
Multi-device	limitato	sì, con CS	sì
Tipico uso	PC, GPS, moduli	sensori, display, memorie	sensori, EEPROM
Complessità	bassa	bassa	media

* in modalità **asynchronous**.

Funzioni

1. HAL_UART_Receive

HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart,uint8_t *pData,uint16_t Size,uint32_t Timeout);

Permette di **ricevere dati dalla USART/UART usando il polling**.

Parametro	Significato
huart	Handle della USART/UART utilizzata
pData	Indirizzo del buffer in cui salvare i dati ricevuti
Size	Numero di byte da ricevere
Timeout	Tempo massimo di attesa

La funzione è **bloccante**: la CPU rimane dentro HAL_UART_Receive() aspettando di ricevere i dati richiesti oppure fino allo scadere del timeout.

Esempio di utilizzo

Un esempio con una stringa di 5 caratteri:

```
uint8_t buffer[5];


HAL_UART_Receive(&huart2, buffer, 5, 1000);
```

La funzione aspetta di ricevere **5 byte**, con timeout di 1000 ms.

2. HAL_UART_Transmit

HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart,const uint8_t *pData,uint16_t Size,uint32_t Timeout);

Permette di **trasmettere dati attraverso la USART/UART usando il polling**.

Parametro	Significato
huart	Handle della USART/UART
pData	Indirizzo dei dati da trasmettere
Size	Numero di byte da trasmettere
Timeout	Tempo massimo di attesa

Anche questa funzione è **bloccante**.

Esempio di utilizzo

```
uint8_t message[] = "Hello World!\r\n";


HAL_UART_Transmit(&huart2,message,sizeof(message) - 1,HAL_MAX_DELAY);
```

Sul terminale del PC vedrai: Hello World!

sizeof(message) - 1 viene utilizzato per non trasmettere il carattere '\0' che termina la stringa C.

3. HAL_UART_Transmit_IT

HAL_StatusTypeDef HAL_UART_Transmit_IT(UART_HandleTypeDef *huart,const uint8_t *pData,uint16_t Size);

Permette di **trasmettere dati tramite UART/USART utilizzando gli interrupt, non è bloccante**.

Quando chiami HAL_UART_Transmit_IT(&huart2, data, Size) la funzione avvia la trasmissione e poi **restituisce il controllo al programma**. La CPU può quindi continuare a fare altro mentre la USART trasmette i dati.

È utile quando:

- non vuoi bloccare il programma durante la trasmissione;
- la CPU deve fare altre operazioni;
- stai gestendo più periferiche contemporaneamente;
- devi trasmettere dati senza occupare continuamente la CPU con il polling.

Esempio di utilizzo

```
uint8_t message[] = "Hello World!\r\n";


HAL_UART_Transmit_IT(&huart2,message,sizeof(message) - 1);
```

Dopo la chiamata, il programma può continuare. Quando la trasmissione è terminata, viene chiamata la callback:

```
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
```

```
    if (huart->Instance == USART2)
    {
        // Trasmissione completata
    }
}
```

4. HAL_UART_Receive_IT

HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart,uint8_t *pData,uint16_t Size);

Permette di **ricevere dati tramite UART/USART utilizzando gli interrupt**.

```
uint8_t data;
HAL_UART_Receive_IT(&huart2, &data, 1);
```

significa: "USART2, quando arriva 1 byte, mettilo in data e avvisami tramite interrupt."

Quando arriva un carattere sulla USART2, viene generato un interrupt e, al completamento della ricezione, viene chiamata:

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        // È arrivato un byte

        // Per esempio, rispediamolo indietro
        HAL_UART_Transmit_IT(&huart2, &data, 1);

        // Riabilitiamo la ricezione
        HAL_UART_Receive_IT(&huart2, &data, 1);
    }
}
```

Attenzione a un dettaglio importante

HAL_UART_Receive_IT() **non significa "ricevi continuamente"**. Se scrivi HAL_UART_Receive_IT(&huart2, &data, 1), hai chiesto: Ricevi **un byte** tramite interrupt. Una volta ricevuto quel byte, la ricezione termina. Se vuoi ricevere nuovamente, devi **riarmare la ricezione**, ad esempio:

HAL_UART_Receive_IT(&huart2, &data, 1);

Questo è il motivo per cui spesso trovi questa funzione dentro HAL_UART_RxCpltCallback().

Confronto con le funzioni di polling

Funzione	Operazione	Metodo	Bloccante?	Callback
HAL_UART_Transmit()	TX	Polling	**Sì**	No
HAL_UART_Receive()	RX	Polling	**Sì**	No
HAL_UART_Transmit_IT()	TX	Interrupt	**No**	HAL_UART_TxCpltCallback()
HAL_UART_Receive_IT()	RX	Interrupt	**No**	HAL_UART_RxCpltCallback()
HAL_UART_Transmit_DMA()	TX	DMA	**No**	HAL_UART_TxCpltCallback()
HAL_UART_Receive_DMA()	RX	DMA	**No**	HAL_UART_RxCpltCallback()

Una distinzione da ricordare bene: con Receive_IT e Receive_DMA devi normalmente specificare **quanti byte (Size) vuoi ricevere**. Se invece non conosci la lunghezza del messaggio, entra in gioco ReceiveToIdle, e in particolare la combinazione molto utile **HAL_UARTEx_ReceiveToIdle_DMA()**.

5. HAL_UARTEx_ReceiveToIdle

HAL_StatusTypeDef HAL_UARTEx_ReceiveToIdle(UART_HandleTypeDef *huart,uint8_t *pData,uint16_t Size,uint16_t *RxLen,uint32_t Timeout);

Questa funzione è particolarmente interessante perché permette di ricevere un numero **variabile di byte**.

"Ricevi dati fino a quando la linea rimane inattiva." Il termine **IDLE** indica quindi che la USART rileva un periodo di inattività sulla linea RX.

RxLen restituisce il **numero effettivo di byte ricevuti**.

Esempio di utilizzo

```
uint8_t buffer[100];
uint16_t RxLen;


HAL_UARTEx_ReceiveToIdle(&huart2,buffer,100,&RxLen,1000);
```

Supponiamo che arrivino:

- HELLO, In questo caso:RxLen = 5;
- buffer: H E L L O ...

Quando usarla?

È molto utile quando ricevi:

- stringhe di lunghezza variabile;
- comandi da un PC;
- pacchetti di lunghezza non nota;

- dati provenienti da moduli esterni.

Nota: questa è ancora una funzione basata sul polling. Esiste anche la variante con interrupt/DMA (HAL_UARTEx_ReceiveToIdle_IT() / ..._DMA()), che è molto utile per applicazioni reali.

6. HAL_UART_Receive_DMA

HAL_StatusTypeDef HAL_UART_Receive_DMA(UART_HandleTypeDef *huart,uint8_t *pData,uint16_t Size);

Questa funzione riceve dati attraverso la UART utilizzando il **DMA — Direct Memory Access**. La differenza fondamentale rispetto a HAL_UART_Receive() è che non è necessario che la CPU gestisca direttamente ogni trasferimento di byte. Il DMA può trasferire automaticamente i dati. La CPU può quindi continuare a eseguire il programma.

Per esempio:

```
uint8_t buffer[100];


HAL_UART_Receive_DMA(&huart2,buffer,100);
```

Significa: "Quando arrivano dati sulla USART2, trasferiscili automaticamente nel buffer, fino a 100 byte."

Quando usarla?

È particolarmente utile quando:

- arrivano molti dati;
- la comunicazione è continua;
- non vuoi bloccare la CPU;
- devi acquisire dati ad alta velocità;
- vuoi usare la UART insieme ad altre periferiche.

Esempio di utilizzo

```
uint8_t rxBuffer[20];


HAL_UART_Receive_DMA(&huart2,rxBuffer,20);
```

Il DMA riempirà:

rxBuffer[0]

rxBuffer[1]

rxBuffer[2]

...

rxBuffer[19]

man mano che arrivano i dati. Al completamento del trasferimento puoi gestire l'evento tramite callback, ad esempio:

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        // 20 byte ricevuti
    }
}
```

7. HAL_UART_Transmit_DMA

HAL_StatusTypeDef HAL_UART_Transmit_DMA(UART_HandleTypeDef *huart,const uint8_t *pData,uint16_t Size);

È la versione DMA della trasmissione. il DMA trasferisce automaticamente i dati dalla **RAM al registro della USART**. La CPU può quindi continuare ad eseguire il resto del programma mentre la trasmissione avviene.

Esempio di utilizzo

```
uint8_t message[] = "Hello World!\r\n";


HAL_UART_Transmit_DMA(&huart2,message,sizeof(message) - 1);
```

La funzione avvia la trasmissione e **ritorna subito**; non aspetta che tutti i caratteri siano stati fisicamente trasmessi.

Al termine puoi usare:

```
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        // Trasmissione completata
    }
}
```

Confronto finale

Funzione	Direzione	Metodo	Bloccante?	Numero dati
HAL_UART_Receive()	RX	Polling	**Sì**	Noto
HAL_UART_Transmit()	TX	Polling	**Sì**	Noto
HAL_UARTEx_ReceiveToIdle()	RX	Polling	**Sì**	**Variabile**
HAL_UART_Receive_DMA()	RX	DMA	**No**	Noto
HAL_UART_Transmit_DMA()	TX	DMA	**No**	Noto

Progetti

Lab04_1a

Lab04_1a: Inviare dal micro al PC ogni secondo una stringa con nome, cognome e data di nascita con il segnale di accapo.

File .ioc

Pin

- PA2: USART2_TX
- PA3: USART2_RX

Timer

- Timer 2:
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1

Codice

Private Include

```
#include "string.h"
```

Private Variables

```
uint8_t string[]="Fabiola Ribolsi 15/08/2002\n";
```

Interrupt Timer 2

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
	if(htim->Instance == TIM2){
		HAL_UART_Transmit(&huart2,string,sizeof(string) - 1,HAL_MAX_DELAY);
	}
}
```

Inizializzazione Timer 2

```
HAL_TIM_Base_Start_IT(&htim2);
```

Lab04_1b

Lab04_1b: rifare Lab04_1a con DMA.

File .ioc

Pin

- PA2: USART2_TX
- PA3: USART2_RX

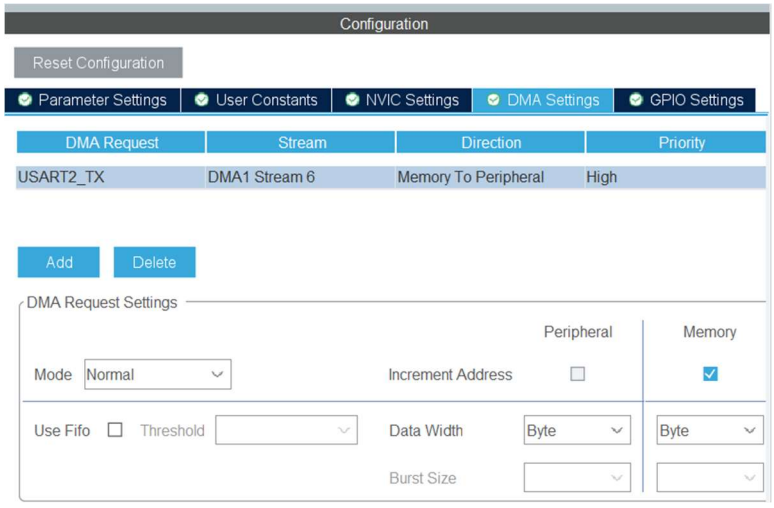
Timer

- Timer 2:
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC: USART2 global interrupt = Enabled



Codice

Private Include

```
#include "string.h"
```

Private Variables

```
uint8_t string[]="Fabiola Ribolsi 15/08/2002\n";
```

Interrupt Timer 2

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
	if(htim->Instance == TIM2){
		HAL_UART_Transmit_DMA(&huart2,string,sizeof(string) - 1);
	}
}
```

Inizializzazione Timer 2

```
HAL_TIM_Base_Start_IT(&htim2);
```